



Ankara Üniversitesi Bilgisayar Mühendisliği
BLM 4062 – Araştırma Teknikleri 2

Ad: Zeynep Sude ÖNCÜ, Semih CENGİZ

Numara: 22290412, 20290233

Danışman : Sevgi Yiğit SERT

Proje Başlığı: DeepSearch with LLM

1. Giriş

Bu belge, DeepSearch yazılım projesinin yapım süreci boyunca izlenecek test sürecine dair temel yol haritasını sunmak amacıyla hazırlanmış test dokümanıdır. Bu doküman projenin, belirlenen fonksiyonel ve fonksiyonel olmayan gereksinimleri eksiksiz bir şekilde karşıladığından emin olmak amacıyla hazırlanmıştır.

1.1. Amaç

Bu belgenin amacı, DeepSearch projesinin parçalarının proje gereksinimlerine uygun olup olmadığını kontrol etmek için planlanan test sürecini açıklamaktır. Aynı zamanda proje kapsamında yapılacak test çalışmaları boyunca referans belge olarak kullanılmak amacıyla hazırlanmıştır.

Bizim bu rapordaki test ettiğimiz genel şeylere bakacak olursak “Sistem kullanıcı sorularına cevapları doğru dokümandan getiriyor mu?”, “Getirilen dokümanlar soruyla ilgili mi?”, “Halüsinasyon mekanizması doğru mu?” gibi sorulara cevap üretmek amacıyla yazılmıştır.

1.2. Kapsam

Bu test planı, DeepSearch projesinin aşağıda listelenen ana bileşenlerini ve bu bileşenler arasındaki entegrasyonu kapsamaktadır.

Test Kapsamındaki Alanlar:

- **Frontend (Next.js):** Kullanıcı arayüzü bileşenleri, kullanıcı etkileşimleri, arama sorgusu gönderimi ve sonuçların/raporların görüntüleyip görüntüleyemediğini test ettik..

- **Backend (Express.js):** Frontend'ten gelen isteklerin karşılandığını ve Python-Core tarafına yönlendirilmesi uygun formatta olup olmadığı test edildi.

- **Python-Core (RAG Sistemi):** Sistemimizin ana bölümü olup dokümanların işlenmesi, vektör temsilleri oluşturulması, anlamsal arama yapılması, Büyük Dil Modeli ile cevap üretilmesi ve hatalı sonuçların kontrol edilmesi test edildi.

1.3. Tanımlar, Kısaltmalar ve Kısaltmalar

Terim	Açıklama	
RAG	LLM'in yanıt üretmeden önce bir bilgi kaynağından bağlam topladığı hibrit yaklaşım. Arama (retrieval) ve üretim (generation) adımlarını birleştirir.	Python-Core
LLM	Doğal dil işleme görevlerinde kullanılan, büyük veri kümeleri üzerinde eğitilmiş yapay zeka modeli. Metin üretme, özetleme, soru yanıtlama gibi işlemleri gerçekleştirebilir.	Python-Core
Embedding	Metin, kelime veya belge gibi verilerin çok boyutlu bir sayısal vektör olarak temsil edilmesi. Semantik benzerlik hesaplamalarında kullanılır.	Python-Core
Vektör Veritabanı	Embedding vektörlerini depolayan ve yaklaşık en yakın komşu (Approximate Nearest Neighbor - ANN) araması gibi verimli benzerlik arama algoritmalarını destekleyen özel veritabanı.	Python-Core

1.4. Referanslar

LangChain, *LangChain Documentation*. [Çevrimiçi]. Erişim: <https://docs.langchain.com/> • Chroma, *ChromaDB Documentation*. [Çevrimiçi]. Erişim: <https://docs.trychroma.com/> • IEEE Standards Association, *ISO/IEC/IEEE 29148:2018 — Systems and Software Engineering — Requirements Engineering*, 2018.

1.5. Belge Yapısına Genel Bakış

Bu test belgesi, aşağıdaki bölümlerden oluşmaktadır:

- **Bölüm 2 - Test Öğeleri:** Test edilecek yazılım bileşenlerini ve doğrulanacak işlevleri tanımlar.
- **Bölüm 3 - Test Stratejisi:** Test seviyeleri, türleri ve süreç yönetimi kriterleri dahil olmak üzere genel test yaklaşımını özetler.
- **Bölüm 4 - Test Tasarımı ve Spesifikasyonu:** Belirlenen işlevler için yürütülebilir, ayrıntılı test senaryolarını içerir.
- **Bölüm 5 - Test Ortamı Kurulumu:** Testlerin yapılacağı bilgisayarlar, programlar ve diğer araçların neler olması gerektiğini açıklar.
- **Bölüm 6 - Test Takvimi:** Test faaliyetlerinin proje zaman çizelgesindeki yerini ve kilometre taşlarını tanımlar.
- **Bölüm 7 - Test Çıktıları:** Test süreci sonunda üretilecek raporları ve belgeleri listeler.
- **Bölüm 8 - Ekler:** Destekleyici ek bilgi ve materyalleri içerir.

2. Test Öğeleri

Bu bölüm, DeepSearch projesinde kullanılan test edilecek bileşenlerin neler olduğunu ve bu bileşenlerin özellikleri ve işlevleri hakkında bilgi vermektedir. Projemiz frontend, backend ve Python-Core olmak üzere üç ana bölümden oluşmuştur.

2.1. Test Edilecek Bileşenler/Modüller

2.1.1. Frontend (Next.js Kullanıcı Arayüzü)

Sistem kullanıcılarının uygulama ile etkileşime geçtiği bölüm olup bu bölümde kullanıcıların doğal dilde soru sormasını, sistemden geri dönüt aldığı ve ilgili dokümanlara ulaştığı kısımdır.

Biz sistemimizin testini yaparken “Arama sorgu giriş alanını” , “ Sorgu gönderme butonunu” , “Cevap görüntülemeyi” , “Hata durumunda gelen hata mesajlarını” ve “Rapor ve sonuç durumu” test edilen temel frontend bileşenleridir.

Aynı zamanda yazılan sorgunun backend’e doğru gönderilmesi de test bileşenlerimiz arasında yer almaktadır.

2.1.2. Backend (Express.js)

Backend sistemimiz bizim projemizde köprü görevi gören yapıdır. Kullanıcıdan gelen HTTP isteklerini alıp Python-Core’a yönlendirir ve Python-Core’dan dönen mesajı da frontende göndermekle yükümlüdür.

- **API Uç Noktaları:** Frontend'den gelen HTTP isteklerinin (GET/POST) karşılanması, bu isteklere uygun yanıtın döndürülmesi.

- **Entegrasyon Katmanı:** Kullancıdan alınan sorguların Python-Core servisine iletilmesi ve oradan dönen RAG tabanlı yanıtların işlenerek üretilen yanıtların kullanıcıya anlaşılır bir şekilde geri döndürülmesi.

2.1.3. Python-Core (RAG Sistemi)

Tüm sistemin doğruluğu test edilecektir. Buna vektör veri tabanının birimleri getirme yeteneği, tüm graph modülündeki zincirler ve modüller dahildir. Diğer başlıklara geçmeden önce kullanılan test modülünden bahsetmek istiyorum.

Test verisi olarak SQuAD 2.0 kullanılmıştır. Bu veriler Wikipedia paragrafları üzerinden hazırlanmış soru-cevap kayıtlarından oluşmaktadır.

```
{
  "answers": [
    {
      "answer_start": 704,
      "text": "Jingshi Dadian"
    },
    {
      "answer_start": 704,
      "text": "Jingshi Dadian"
    },
    {
      "answer_start": 704,
      "text": "Jingshi Dadian"
    }
  ],
  "id": "5728710c3acd2414000df9f2",
  "is_impossible": false,
  "question": "What was the most notable publication of Tugh's academy?"
},
```

Şekil 1 Test Verisinden Bir Örnek

Her kayıt şu bilgileri içerir:

- Sorunun bağlı olduğu referans paragraf
- Soru metni
- Kabul edilen kısa cevaplar
- Sorunun cevaplanabilir olup olmadığı
- Cevaplanamaz sorular için yanıltıcı olabilecek plausible answer bilgileri

Sistem toplam 1024 doküman arasından ve her doküman içindeki cümlelerden cevabı bulmaya çalışmaktadır. Bu kadar fazla doküman olması gerçekte kullanılabilecek senaryoyu yansıtmaktadır.

2.2. Doğrulanacak Özellikler ve İşlevler

Özellik/İşlev	İlgili Bileşen(ler)	Doğrulama Amacı
Kullanıcı Arama Sorgusu Gönderimi	Frontend, Backend	Kullanıcının arayüzden girdiği sorunun backend tarafına başarıyla iletilildiğini doğrulamak.
Sorguya Göre Doküman Getirme	Python-Core (graph/nodes, graph/chains)	Kullanıcı sorgusuna en uygun dokümanların vektör veritabanından doğru bir şekilde getirildiğini analiz etmek.
LLM ile Cevap Üretimi	Python-Core (graph/nodes, graph/chains)	Getirilen dokümanlardaki bilgiler kullanılarak, kullanıcıya sorgusuyla alakalı doğru ve tutarlı bilgiler getirdiğimizi doğrulamak
Halüsinasyon Kontrolü	Python-Core (graph/chains)	Üretilen cevabın, kaynak dokümanlarda bulunmayan bilgiler içermediğini (halüsinasyon) denetlemek.
Sonuçların Rapor Halinde Sunulması	Backend, Python Core, Frontend	Verilen cevapta, ilgili dokümanlarla oluşturulmuş özet cevabın, sorunun dokümanın içeriğindeki yerin(sayfa bilgisi vb.) doğrulanması ve bunların rapor halinde kullanıcıya sunulması.

3. Test Stratejisi

Bu bölüm test sürecinin nasıl yapılacağını ve işleneceğini gösterir. DeepSearch projesini hedeflerine ulaşmak için izlenecek test seviyeleri, test türleri, giriş çıkış kriterleri ve askıya alma yeniden başlama kriterlerini ortaya koyar.

3.1. Test Seviyeleri

Test faaliyetleri dört ana başlıkta gerçekleşir:

Birim Testi: Yazılımın en küçük, test edilebilir parçalarının (fonksiyonlar, metotlar, sınıflar) ayrı ayrı şekilde test edilmesidir. Amaç, her bir kod parçasının beklendiği gibi çalıştığını görmektir.

Örnek: retrieval_grader.py içindeki _grade_single_doc fonksiyonu kendisine gelen her bir dokümanın soruyla alakalı olup olmadığını anlamaya çalışmaktadır. Bunu her bir gelen her bir doküman için LLM çağrısı yaparak sağlamaktadır. Bunun sonucunu terminale yazarak geliştiricinin, sistemin hangi dokümanları seçtiğini görmesini sağlamaktadır.

```
async def _grade_single_doc(doc, question: str, index: int):
    """
    Single document grading - runs concurrently.
    """
    score = await retrieval_grader.ainvoke(
        {"question": question, "document": doc.page_content}
    )
    grade = score.binary_score.lower()
    status = "RELEVANT" if grade == "yes" else "NOT RELEVANT"
    print(f"---GRADE [{index + 1}]: DOCUMENT {status}---")

    return doc if grade == "yes" else None
```

Şekil 2 Grade Single Doc Fonksiyonu

```
system = """
You are a grader assessing relevance of a retrieved document to a user question. \n
If the document contains keyword(s) or semantic meaning related to the question, grade it as relevant. \n
Give a binary score 'yes' or 'no' score to indicate whether the document is relevant to the question.
"""
```

Şekil 3 Sistemin Karar Vermesi İçin Gereken Prompt

```
---GRADE [19]: DOCUMENT NOT RELEVANT---
---GRADE [17]: DOCUMENT NOT RELEVANT---
---GRADE [4]: DOCUMENT NOT RELEVANT---
---GRADE [14]: DOCUMENT NOT RELEVANT---
---GRADING COMPLETE: 1/20 documents passed---
```

Şekil 4 Doküman Alakası Kararından Terminal Çıktısı

Entegrasyon Testi: Farklı modüllerin veya bileşenlerin bir araya getirildiğinde uyumlu bir şekilde çalışıp çalışmadığını doğrular.

Örnek: Backend ile Python-Core arasındaki entegrasyon, kullanıcının sorduğu sorunun backend üzerinden Python-Core API'ye iletilmesiyle test edilmiştir. chat.routes.js dosyasında /api/chat/ask endpoint'i tanımlanmıştır. Bu endpoint önce kullanıcı kimliğini authenticate middleware'i ile doğrulamakta, ardından chat.controller.js üzerinden askQuestionForUser servisini çağırılmaktadır. chat.service.js içinde kullanıcının erişebileceği dokümanlar alınmakta ve bu doküman adları allowed_sources olarak Python-Core tarafındaki /query endpoint'ine gönderilmektedir. Python-Core tarafında api.py dosyası bu isteği karşılamakta ve answer_question_with_scoped_documents fonksiyonunu çalıştırmaktadır. Bu test ile backend'in Python-Core sistemini doğru formatta tetiklediği ve Python-Core'dan dönen cevabı tekrar frontend'e uygun formatta döndürdüğü doğrulanmıştır.

Sistem Testi: Bir sistemin tüm gereksinimleri karşılayıp karşılamadığını görmek için yapılan testlerdir. Bu testler, sistemin baştan sona tüm bölümlerini kapsar.

Örnek: Kullanıcının arayüze bir arama sorgusu girmesinden başlayarak, RAG sisteminin çalışması, cevabın üretilmesi ve sonucun rapor olarak kullanıcı ekranında gösterilmesine kadar olan tüm akışın test edilmesi.

```
workflow = StateGraph(GraphState)

# =====
# NODES
# =====
workflow.add_node(RETRIEVE, retrieve)
workflow.add_node(GRADE_DOCUMENTS, grade_documents)
workflow.add_node(GENERATE, generate)
workflow.add_node(FALLBACK_TO_LLM, fallback_to_llm)

# =====
# ENTRY POINT: Direkt RETRIEVE (router yok)
# =====
workflow.set_entry_point(RETRIEVE)
```

Şekil 5 WorkFlow Yapısından Bir Örnek

Kullanıcı Kabul Testi: Sistemin son kullanıcıların veya müşterilerin ihtiyaç ve beklentilerini karşılayıp karşılamadığını doğrulamak amacıyla yapılan testlerdir.

Örnek: Savunma sanayi şirketindeki bir kullanıcının, sistemde olan yüzlerce doküman arasından, kendi sorgusuna göre üretilen cevabı arayüzde görebilmesi. Ayrıca bu sonuç doğru ve tutarlı olmalıdır ki kullanıcı yazılımdan memnun kalsın.

3.2. Test Türleri

Belirlenen test seviyelerinde aşağıdaki test türleri uygulanacaktır:

İşlevsel Testler: Sistemin ne yaptığını test eder. Arama, doküman getirme, cevap üretme ve raporlama gibi temel fonksiyonların, doğru çalışıp çalışmadığını test eder.

İşlevsel olmayan testler: Sistemin nasıl çalıştığını değerlendirir. Bu proje kapsamında özellikle aşağıdaki alanlara odaklanılacaktır:

- **Performans:** Sisteme paralel çalışma özelliği ekleyerek aynı anda birden fazla LLM çağrısı yapılması sağlanmıştır. Örneğin getirilen birimlerin alakalı olup olmadığı eskiden tek tek kontrol ediliyordu. Bir birim için LLM işini bitirdiğinde, diğer birim için LLM çağrılıyordu. Şimdi işlemler paralel yapılarak aynı anda birden fazla birim kontrol ediliyor. Bu sayede bu bölümün çalışma süresi 1/8 oranına düşmüştür. Bu ciddi süre tasarrufu toplam sistemin çalışma süresini 1/3 oranında daha hızlı çalışmasına olanak tanımıştır. Her bir sorunun çalışma süresinin sisteme ve doküman yoğunluğa bağlı olarak değişmekle birlikte, 1024 doküman olan sistemde Apple M4 Pro işlemci ile yaklaşık 20sn sürmektedir.
- **API:** Sistemimizde birden fazla katman yer almaktadır. Bu ana katmanlar frontend, backend ve Python-Core katmanlarıdır. Sistem katmanlar arasındaki veri alışverişi ile ilerlemektedir. API testi de bu veri alışverişlerinin doğru şekilde yapılıp yapılmadığı ile ilgilidir. Bizim sistemimizdeki backend katmanı frontend ve Python-Core arasında köprü görevi görmektedir. API testlerinde dikkat ettiğimiz nokta frontend tarafından kullanıcının girmiş olduğu sorgunun backendte ulaşp, backend bunu doğru endpoint üzerinden Python-Core'a gönderip gönderemediğidir. Sorgu sorma akışında /api/chat/ask endpoint'i test edildi. Bu endpoint kullanıcının gönderdiği question bilgisi alır.
- **Kullanılabilirlik:** Sistem kullanıcıları tarafından kolay, anlaşılır ve hatasız bir şekilde kullanıldığını test edilmek amacıyla yapılmıştır. Bu değerlendirme bir teknik sistem değerlendirmesinden ziyade kullanıcılar sisteme girdiğinde anlaşılmasının kolay olup olmadığını tespit etmek için yapılmıştır. Bunun için hata mesajlarının net ve anlaşılır olması da önem arz etmektedir.

3.3. Giriş ve Çıkış Kriterleri

Test sürecinin başlangıç ve bitiş noktalarını belirleyen net kriterler aşağıda tanımlanmıştır:

- **Giriş Kriterleri:** Test faaliyetlerinin başlayabilmesi için sağlanması gereken koşullar:
 - o Test edilecek modülün veya özelliğin kodlamasının tamamlanmış ve derlenmiş olması.
 - o Geliştiriciler tarafından yazılan temel birim testlerinin başarıyla geçmesi.
 - o Test ortamının ve gerekli araçların hazır olması.
- **Çıkış Kriterleri:** Test sürecinin tamamlanmış sayılması içingereken koşullar:
 - o Tanımlanan tüm test senaryolarının yürütülmüş olması.
 - o Test senaryolarının çoğunluğunu başarı ile tamamlamış olmak
 - o Tespit edilen tüm kritik ve yüksek öncelikli hataların çözülmüş olması.

4. Test Tasarımı ve Spesifikasyonu

Gereksinim ID	Gereksinim Açıklaması	Gereksinim Türü	Açıklama / Kapsam
REQ-001	Kullanıcı girişi Sistem kullanıcıların doküman yüklemesine izin vermelidir	Fonksiyonel	Kullanıcılar PDF ve TXT formatındaki dokümanları sisteme yükleyebilmelidir
REQ-002	Sistem yüklenen dokümanları metin parçalarına ayırmalıdır	Fonksiyonel	Kullanıcı hesabı mevcut olmalıDokümanlar LLM ve embedding için uygun parçalara bölünmelidir
REQ-003	Sistem metin parçaları için embedding üretmelidir	Fonksiyonel	Her metin parçası vektör temsiline dönüştürülmelidir
REQ-004	Sistem embedding'leri vektör veritabanında saklamalıdır	Fonksiyonel	Anlamsal arama için kalıcı depolama sağlanmalıdır
REQ-005	Sistem kullanıcı sorgularına anlamsal arama yapabilmelidir	Fonksiyonel	Vektör benzerliğine dayalı retrieval gerçekleştirilmelidir
REQ-006	Sistem LLM tabanlı cevap üretmelidir	Fonksiyonel	Retrieval sonuçlarına dayalı doğal dilde cevap üretilmelidir
REQ-007	Sistem internet bağlantısı olmadan çalışabilmelidir	Fonksiyonel Olmayan	Tüm işlemler yerel ortamda yapılmalıdır
REQ-008	Sistem yetkisiz erişimleri engellemelidir	Fonksiyonel Olmayan	Kullanıcı doğrulama ve erişim kontrolü sağlanmalıdır

Tablo 1- Sistem Gereksinim Tablosu

Test ID	Test Açıklaması	Gereksinim ID	Ön Koşullar	Giriş Verileri	Beklenen Sonuçlar
TEST-001	Doküman yükleme ve indeksleme testi	REQ-001, REQ-002, REQ-003, REQ-004	Sistem çalışır durumda olmalı, vektör veritabanı aktif olmalı	PDF formatında doküman	Doküman başarıyla yüklenir, parçalanır, embedding üretilir ve vektör veritabanına kaydedilir
TEST-002	Desteklenmeyen dosya formatı testi	REQ-001	Sistem çalışır durumda olmalı	.exe veya .zip dosyası	Dosya reddedilir ve kullanıcıya hata mesajı gösterilir
TEST-003	Metin parçalama doğruluk testi	REQ-002	Doküman sisteme yüklenmiş olmalı	Uzun metin içeren doküman	Metin belirlenen sınırlar dahilinde parçalara ayrılır
TEST-004	Embedding üretim tutarlılığı testi	REQ-003	Embedding modeli aktif olmalı	Metin parçası	Her parça için doğru boyutta embedding üretilir
TEST-005	Vektör veritabanı kayıt ve sorgulama testi	REQ-004, REQ-005	Embedding'ler veritabanında kayıtlı olmalı	Kullanıcı sorgusu	En alakalı metin parçaları başarıyla döndürülür
TEST-006	Retrieval doğruluk testi	REQ-005	Vektör veritabanı dolu olmalı	Anlamsal kullanıcı sorgusu	Alakalı sonuçlar listelenir
TEST-007	LLM cevap üretim testi	REQ-006	Retrieval süreci başarılı olmalı	Kullanıcı sorgusu	LLM, retrieval içeriğine dayalı tutarlı cevap üretir
TEST-008	Halüsinasyon kontrol testi	REQ-006	Dokümanlar indekslenmiş olmalı	Belirli içerik sorgusu	Yanıt doküman dışı bilgi içermez
TEST-009	Offline çalışma testi	REQ-007	İnternet bağlantısı kapalı olmalı	Kullanıcı sorgusu	Sistem offline durumda çalışmaya devam eder
TEST-010	Yetkisiz erişim	REQ-008	Kullanıcı yetkisiz	Yetkisiz sorgu	Erişim reddedilir ve

Tablo 2- Test Tasarımı ve Spesifikasyonu Tablosu

5. Test Ortamı Kurulumu

Bu bölüm, testlerin tutarlı ve güvenilir vermesi için gereken donanım, yazılım ve araç altyapısını tanımlar. Test sonuçlarının kalitesinin çalıştırılacak ortamın farklılıklarına göre değişmemesi gerekir fakat sonuçların kullanıcıya ne kadar sürede iletileceği (hızı) donanıma bağlı olarak değişebilir.

5.1. Donanım ve Yazılım Yapılandırmaları

Testlerin yürütüleceği ortam için minimum gereksinimler aşağıda listelenmiştir:

- Donanım: Standart bir geliştirici bilgisayarı:
 - RAM: Minimum 16 GB
 - Kabul edilebilir bir bekleme süresi için CPU ve GPU beraber kullanılmalıdır: Minimum gtx 1650ti veya muadili ve i5 9.nesil intel işlemci veya muadili. MacOS bilgisayarlar için M1 silicon veya üzeri önerilir.
 - Depolama: LLM Modelleri yaklaşık 10GB yer kaplamaktadır. Kullanılan dokümanların büyüklüğüne göre topla kapasite belirlenmelidir.
- İşletim Sistemi: Windows 10/11, macOS veya Linux (Ubuntu 20.04+).
- Yazılım:
 - Frontend ve Backend: Node.js (LTS versiyonu), npm/yarn paket yöneticisi.
 - Bilgisarda mutlaka Ollama yüklü olmalıdır.
 - Python-Core: Python 3.13
 - Python Kütüphaneleri: requirements.txt dosyasında belirtilen diğer tüm bağımlılıklar.
 - Veritabanı: Projede kullanılan vektör veritabanı için gerekli yerel kurulum.

5.2. Gerekli Araçlar ve Simülatörler

Test sürecinin verimliliğini artırmak ve otomatize etmek için aşağıdaki araçlar kullanılacaktır:

- **Birim Test Araçları:**
 - **Frontend:** Jest
 - **Backend:** Mocha ve Chai
 - **Python-Core:** Pytest
- **API Test Araçları:** Backend API endpoint'lerini manuel olarak test etmek ve otomasyon betikleri oluşturmak için Postman.
- **Sürüm Kontrolü:** Projeyi yönetmek ve değişiklikleri takip etmek için Git.
- **Hata Takibi:** Tespit edilen hataları kaydetmek, önceliklendirmek ve yaşam döngüsünü takip etmek için GitHub Issues.

6. Test Takvimi

Aktivite	Başlangıç Tarihi	Bitiş Tarihi	Sorumlular
Test Planlaması	26 Mayıs 2026	30 Mayıs 2026	Semih Cengiz Zeynep Sude Öncü
Birim ve Entegrasyon Testleri	1 Haziran 2026	3 Haziran 2026	Semih Cengiz
Sistem Testleri	4 Haziran 2026	7 Haziran 2026	Zeynep Sude Öncü
Hata Düzeltme ve Regresyon Testleri	8 Haziran 2026	9 Haziran 2026	Semih Cengiz Zeynep Sude Öncü
Son Test Raporunun Hazırlanması	10 Haziran 2026	12 Haziran 2026	Semih Cengiz Zeynep Sude Öncü

7. Test Çıktıları

Bu bölüm, test sürecinin sonuçlarını ve bulgularını belgeleyen önemli çıktıları açıklar. Bu çıktılar, projenin mevcut kalitesi hakkında somut bilgiler verir.

Test süreci sonunda üretilmesi beklenen temel çıktılar şunlardır:

- **Test Senaryosu Yürütme Raporu:** Yürütülen her bir test senaryosunun sonucunu içeren, yapılacak testlerin ne kadarının tamamlandığını gösteren bir rapordur. Örnek olarak SQuAD 2.0 test sonuçları verilebilir.

```
"summary": {  
  "evaluated_questions": 1000,  
  "answerable_questions": 956,  
  "impossible_questions": 44,  
  "pass_rate": 0.86,  
  "exact_match": 0.0,  
  "token_f1": 0.31,  
  "contains_gold_rate": 0.88  
},
```

Şekil 6 Test Sonuçları

Burada sisteme toplam 1000 tane soru sorulmuştur. Sorulardan 956 tanesinin test verisi içinde cevabı bulunmaktadır, 44 tanesinin ise cevabı bulunmamaktadır. Şekil 00 da gösterilen resim içinde 'is_impossible' anahtarı bu bilgiyi vermektedir. Diğer metriklerin anlamı şu şekildedir:

- **Pass Rate:** Toplam soruların ne kadarının başarılı kabul edildiğini gösterir. Sistem toplam soruların %86'sını doğru cevaplamıştır.
- **Exact Match:** Üretilen cevabın referans cevapla tamamen aynı olup olmadığını ölçer.

Testte Exact Match değerinin 0.0 olması, hiçbir cevabın doğru olmadığı anlamına gelmez. Model genellikle kısa cevap yerine tam cümle oluşturarak cevap vermektedir. Zaten bu testte tam olarak aynı cevap vermesi beklenmemektedir. Önemli olan anlamsal doğruluktur.

- **Token F1:** Üretilen cevap ile referans cevap arasındaki ortak kelimeleri ölçer. Bu değer Pass Rate'e göre düşük olmasının temel nedeni, modelin referans cevaplardan daha uzun ve açıklamalı cevaplar üretmesidir.
- **Contains Gold Rate:** Referans cevabın normalize edilmiş hâlinin model cevabı içinde bulunup bulunmadığını ölçer. Aslında doğruluk olarak bu değeri referans olmak daha gerçekçidir.

Bu test sonuçlarını daha iyi anlamak için bazı örnekler vermek gerekirse; 'is_impossible', 'true' olduğunda bu sorunun sistem içinde cevabı olmadığını göstermektedir ve sistemin bu sorulara "I don't know" şeklinde cevap vermesi beklenmektedir.

```
"cases": [  
  {  
    "id": "5a82025a31013a001a3350b9",  
    "question": "What denomination was the Hollis Professor of Divinity connected to?",  
    "is_impossible": true,  
    "gold_answers": [],  
    "answer": "I don't know.",  
    "retrieved_ids": [  
      "20-5"  
    ],  
    "gold_passage_id": "20-5",  
    "retrieval_hit": 1.0,  
    "generation_retries": 2,  
    "source_count": 0,  
    "quality_rejected": true,  
    "rejection_reason": "hallucination_after_max_retries",  
    "exact_match": 1.0,  
    "token_f1": 1.0,  
    "contains_gold": 1.0,  
    "no_answer_correct": 1.0,  
    "passed": true  
  }  
]
```

Şekil 7 İmkansız Soruda Sistemin Cevabı

Şekilde görüldüğü gibi imkansız soruya sistem “I don’t know” cevabını vermiştir. Sorunun cevabı olabilecek bir doküman yine de veri setinde bulunmaktadır fakat bu yanıltıcı bir dokümandır. Sistem o dokümanı bulmuş ama cevapta halüsinasyon olduğunu fark etmiş ve bilmiyorum şeklinde yanıt vermiştir.

Şekil 8’de görüldüğü gibi sistem bu soruya cevap üretememektedir. Buradaki sorun vektörel veri tabanının doğru birimleri getirememesinden kaynaklanmaktadır. Getirilen hiçbir doküman olmadığından sistem bilmiyorum şeklinde yanıt vermiştir. Bu yüzden cevap yanlış olarak kabul edilmiştir.

```
{
  "id": "56de0ffd4396321400ee258f",
  "question": "Who ruined Roussel de Bailleul's plans for an independent state?",
  "is_impossible": false,
  "gold_answers": [
    "Alexius Komnenos",
    "Alexius Komnenos",
    "Alexius Komnenos"
  ],
  "answer": "I don't know.",
  "retrieved_ids": [],
  "gold_passage_id": "0-8",
  "retrieval_hit": 0.0,
  "generation_retries": 2,
  "source_count": 0,
  "exact_match": 0.0,
  "token_f1": 0.0,
  "contains_gold": 0.0,
  "no_answer_correct": null,
  "passed": false
},
```

Şekil 8 Üretilen Yanlış Cevap

- **Hata Raporları:** Tespit edilen her bir hata için ayrı ayrı oluşturulan standart raporlardır. Bu raporlar; hatanın adını, ne olduğunu, olması gereken ve olan sonuçları, hatanın önceliğini (critical, high, medium, low) ve mevcut durumunu (new, in progress, resolved, closed) içerir. Bu raporlar, Bölüm 5.2'de belirtilen Hata Takibi aracında yönetilecektir.
- **Test Özet Raporu :** Test aşamalarının sonunda hazırlanır. Test edilen işlevleri, test sonuçlarını ve bulunan hataları içerir.
- **Günlük Dosyaları:** Hataları anlamak ve nereden kaynaklandığını bulmak için kullanılır. Test sırasında backend ve python-core tarafından oluşturulan detaylı kayıtlardır.

```
RETRIEVE FROM VECTOR DATABASE
Question: What denomination was the Hollis Professor of Divinity connected to?
Similarity Search Results:
Retrieved: 20 documents

1. ✓ Distance: 0.285
File: 20-5

2. ✓ Distance: 0.293
File: 20-7

3. ✓ Distance: 0.343
File: 5-35

4. ✓ Distance: 0.350
File: 19-3

5. ✓ Distance: 0.353
File: 20-4

6. ✓ Distance: 0.354
File: 20-0

7. ✓ Distance: 0.370
File: 23-35

8. ✓ Distance: 0.370
File: 5-22

9. ✓ Distance: 0.378
File: 15-24

10. ✓ Distance: 0.380
File: 23-33
```

Şekil 9 Anlık Test Edilen Terminal Çıktısına Örnek - 1

```

11. ✓ Distance: 0.382
File: 20-6

12. ✓ Distance: 0.383
File: 5-2

13. ✓ Distance: 0.385
File: 20-29

14. ✓ Distance: 0.396
File: 23-31

15. ✓ Distance: 0.399
File: 20-3

16. ✓ Distance: 0.399
File: 23-34

17. ✓ Distance: 0.400
File: 23-5

18. ✓ Distance: 0.400
File: 20-28

19. ✓ Distance: 0.401
File: 5-30

20. ✓ Distance: 0.401
File: 23-1
Total retrieved: 20 chunks
Passed cosine distance threshold ( $\leq 0.8$ ): 20 chunks
Unique files: 20

```

Şekil 12 Anlık Test Edilen Terminal Çıktısına Örnek - 2

```

Source Files Used:
• 15-24 (1 chunks)
• 19-3 (1 chunks)
• 20-0 (1 chunks)
• 20-28 (1 chunks)
• 20-29 (1 chunks)
• 20-3 (1 chunks)
• 20-4 (1 chunks)
• 20-5 (1 chunks)
• 20-6 (1 chunks)
• 20-7 (1 chunks)
• 23-1 (1 chunks)
• 23-31 (1 chunks)
• 23-33 (1 chunks)
• 23-34 (1 chunks)
• 23-35 (1 chunks)
• 23-5 (1 chunks)
• 5-2 (1 chunks)
• 5-22 (1 chunks)
• 5-30 (1 chunks)
• 5-35 (1 chunks)

---CHECK THE DOCUMENT RELEVANCE TO QUESTION (PARALLEL)---
---GRADE [3]: DOCUMENT NOT RELEVANT---
---GRADE [2]: DOCUMENT NOT RELEVANT---
---GRADE [7]: DOCUMENT NOT RELEVANT---
---GRADE [1]: DOCUMENT RELEVANT---
---GRADE [16]: DOCUMENT NOT RELEVANT---
---GRADE [8]: DOCUMENT NOT RELEVANT---
---GRADE [10]: DOCUMENT NOT RELEVANT---
---GRADE [5]: DOCUMENT NOT RELEVANT---
---GRADE [14]: DOCUMENT NOT RELEVANT---
---GRADE [17]: DOCUMENT NOT RELEVANT---

```

Şekil 11 Anlık Test Edilen Terminal Çıktısına Örnek - 3

```

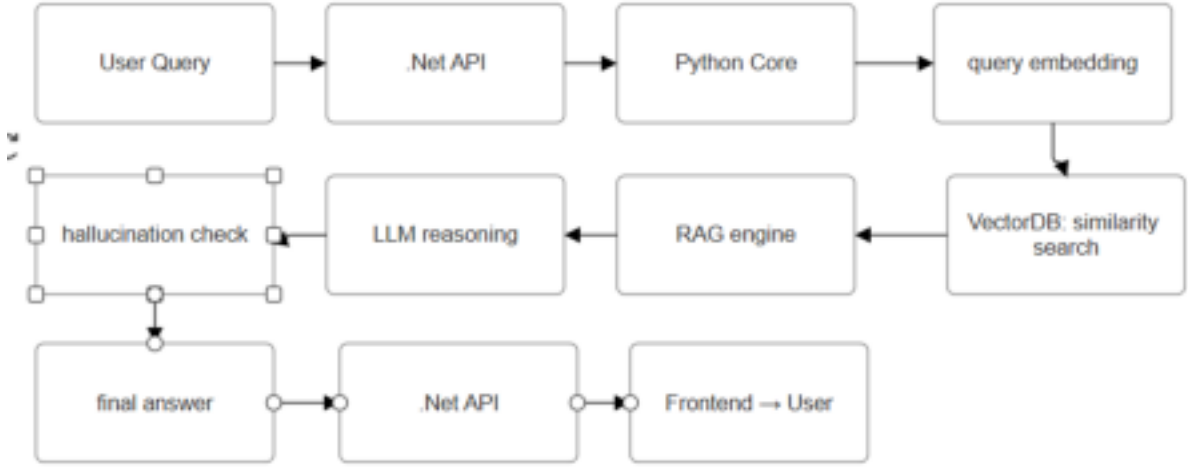
---GRADE [13]: DOCUMENT NOT RELEVANT---
---GRADE [12]: DOCUMENT NOT RELEVANT---
---GRADE [19]: DOCUMENT NOT RELEVANT---
---GRADE [9]: DOCUMENT NOT RELEVANT---
---GRADE [20]: DOCUMENT NOT RELEVANT---
---GRADE [4]: DOCUMENT NOT RELEVANT---
---GRADE [15]: DOCUMENT NOT RELEVANT---
---GRADE [6]: DOCUMENT NOT RELEVANT---
---GRADE [18]: DOCUMENT NOT RELEVANT---
---GRADE [11]: DOCUMENT NOT RELEVANT---
---GRADING COMPLETE: 1/20 documents passed---
---DECIDE AFTER GRADING---
---DECISION: 1 RELEVANT DOCUMENTS → GENERATE---
---GENERATE---
SOURCE: 20-5 | chunk None |
EXCERPT: Throughout the 18th century, Enlightenment ideas of the power of reason and free...
QUALITY CONTROL: CHECKING GENERATION
→ Mode: RAG (checking hallucination)
HALLUCINATION DETECTED → RE-GENERATE (2/2)
---GENERATE---
SOURCE: 20-5 | chunk None |
EXCERPT: Throughout the 18th century, Enlightenment ideas of the power of reason and free...
QUALITY CONTROL: CHECKING GENERATION
→ Mode: RAG (checking hallucination)
HALLUCINATION + MAX RETRIES (2) → REJECT
---REJECT GENERATION: hallucination after max retries---

```

Şekil 10 Anlık Test Edilen Terminal Çıktısına Örnek - 4

8. Ekler

Bu bölüm, belge içinde referans verilen veya ana metni destekleyen ek materyalleri içerir. Okuyucuya test süreçleri ve bulguları hakkında daha derinlemesine bilgi veya görsel kanıtlar sunar. Dokümanın anlaşılabilirliğini artırmayı hedefler.



Şekil 13 Sistemin Genel Yapısı